

Lekcja 08 - Production readiness w Python DE

2026-08-14

Production readiness w Python DE

```
src layout + pytest + mypy + CLI + logging + retry + runbook
```

■ Problem

Skrypt działa lokalnie.

Ale czy ktos inny umie go uruchomic, sprawdzic i naprawic bez pytania autora?

```
Jak podac input?  
Gdzie zapisuje output?  
Gdzie sa logi?  
Jak zrobic rerun?  
Jak testujesz transformacje?
```

To nie jest problem techniczny. To jest problem operacyjny: biznes nie moze zaufac pipeline'owi, jesli nie ma jasnej procedury uruchomienia i debugowania.

Nie chodzi o wielki framework.

Chodzi o minimum porzadku:

```
src/  
tests/  
CLI  
logging  
retry  
README  
runbook
```

■ src layout

```
src/pipeline/  
  cli.py  
  extract.py  
  transform.py  
  load.py  
  
tests/  
  test_transform.py
```

Kod jest importowalny.

Testy nie muszą odpalać całego pipeline'u.

```
python -m pipeline.cli --input data/orders.json --output  
output/result.json
```

CLI oznacza, że nie edytujesz kodu przy każdym runie.

Parametry są jawne.

■ Logging

```
logger.info("Loaded records", extra={"count": len(records)})
```

Log ma odpowiedzic:

```
co wystartowalo  
ile rekordow weszlo  
co zostalo zapisane  
czy byl blad
```

■ Retry

Retry pomaga przy chwilowym błędzie IO.

Nie pomaga przy złym schemacie.

```
API timeout -> retry ma sens  
missing order_id -> retry nie ma sensu
```


■ Testy

Testuj logikę bez prawdziwego scheduler'a.

```
def test_completed_revenue_counts_only_completed() -> None:
    records = [
        {"status": "completed", "total_amount": 100.0},
        {"status": "pending", "total_amount": 50.0},
    ]
    assert calculate_completed_revenue(records) == 100.0
```

Runbook to instrukcja po awarii.

Powinien mieć:

```
jak uruchomic  
jak zrobic rerun  
gdzie sa input/output  
gdzie sa logi  
najczestsze bledy  
owner danych
```

Pytanie:

How do you structure a small production Python ETL project?

Odpowiedz:

I split IO, transformations and loading into modules under src.
I expose the pipeline through a CLI with input and output arguments.
I add tests for pure transformations and validation logic.
The run logs record counts, paths and failures.
For transient IO failures I use bounded retry.
The README and runbook explain how to run and recover the pipeline.

Oddajesz:

```
pyproject.toml  
README.md  
runbook.md  
src/pipeline/  
tests/  
data/orders.json
```

Najwazniejsze:

```
pipeline runs from CLI and tests cover transform logic
```